

Relatório Final XCommerce

Um estudo sobre a transição de uma aplicação de comércio online monolítica para uma abordagem baseada em microsserviços, com recurso ao Spring Boot.

Liandro Macedo da Cruz

PG61472

Miguel Angelo

Second author's affiliation, possibly the same institution, xxxx@gmail.com

Rafael

Third author's affiliation, possibly the same institution, xxxx@gmail.com

Tomás

Third author's affiliation, possibly the same institution, xxxx@gmail.com

A arquitetura de software tem um impacto direto na escalabilidade, manutenção e adaptabilidade das aplicações. As arquiteturas monolíticas constituem uma abordagem tradicional, mas, à medida que as aplicações evoluem e as exigências do mercado se intensificam, surgem desafios como o acoplamento excessivo e a dificuldade de incorporar novas funcionalidades de forma ágil. A migração para uma abordagem baseada em microsserviços promove uma maior modularidade e independência entre os componentes, facilitando a escalabilidade e a resiliência do sistema. Este trabalho analisa a transição de uma aplicação de comércio online, inicialmente desenvolvida como um monólito com Spring Boot, para uma arquitetura distribuída composta por microsserviços. São discutidos os desafios inerentes ao modelo monolítico, os benefícios esperados com a descentralização dos serviços e proposto um modelo arquitetural alinhado com as melhores práticas de engenharia de software distribuído. O objetivo deste estudo é compreender, de forma aprofundada, os desafios e oportunidades envolvidos na mudança de paradigma arquitetural.

CCS CONCEPTS • Arquiteturas de software • Microsserviços • Evolução de software • Desempenho de software

Additional Keywords and Phrases: Arquitetura monolítica, Migração para microsserviços, Escalabilidade, Resiliência, Modernização de software, Engenharia de Software Distribuído

ACM Reference Format:

First Author's Name, Initials, and Last Name, Second Author's Name, Initials, and Last Name, and Third Author's Name, Initials, and Last Name. 2018. The Title of the Paper: ACM Conference Proceedings Manuscript Submission Template: This is the subtitle of the paper, this document both explains and embodies the submission format for authors using Word. In Woodstock '18: ACM Symposium on Neural Gaze Detection, June 03–05, 2018, Woodstock, NY. ACM, New York, NY, USA, 10 pages. NOTE: This block will be automatically generated when manuscripts are processed after acceptance.

1 INTRODUCTION

As arquiteturas de software para sistemas de comércio eletrônico enfrentam um dilema recorrente: à medida que o sistema cresce, o monólito que inicialmente simplificou o desenvolvimento torna-se um obstáculo à evolução independente dos seus domínios funcionais. A literatura de engenharia de software propõe a arquitetura de microserviços como resposta a este problema, mas a transição não é isenta de custos: introduz latência de rede, consistência eventual e complexidade operacional que, em certos contextos, podem superar os benefícios obtidos.

1.1 Problema Arquitetural

Este trabalho estuda empiricamente essa transição a partir de um caso concreto: o **XCommerce**, uma aplicação de comércio eletrônico implementada como monólito em Spring Boot, onde autenticação, catálogo de produtos, carrinho de compras e processamento de encomendas partilham um único processo JVM e uma única instância de base de dados PostgreSQL.

O problema arquitetural central reside no **acoplamento estrutural do fluxo de checkout**: esta operação executa, na mesma transação JPA, a validação do utilizador, a leitura do catálogo, o esvaziamento do carrinho e a criação da encomenda. Daqui resultam dois problemas concretos:

- **Ponto único de falha**: uma degradação em qualquer domínio (por exemplo, sobrecarga no processamento de encomendas) propaga-se imediatamente a todos os restantes, incluindo autenticação e consulta de catálogo.
- **Deployability acoplada**: qualquer alteração ao código de um módulo obriga ao redeployment completo da aplicação, interrompendo temporariamente todas as funcionalidades, independentemente da dimensão da alteração.

A questão central é: **a decomposição em microserviços melhora o isolamento de falhas e a deployability independente por domínio, e a que custo em termos de latência e complexidade operacional?**

1.2 Alternativas Arquiteturais Consideradas

Para resolver este problema foram consideradas três abordagens:

Monólito Modular. Introdz separação lógica entre módulos mantendo um único artefacto de deployment. Reduz o acoplamento de código mas não elimina o ponto único de falha operacional nem permite scaling ou redeployment independente por domínio.

SOA com ESB. Decompõe o sistema em serviços integrados por um Enterprise Service Bus centralizado, responsável pelo roteamento, transformação de mensagens e orquestração de workflows. O problema desta abordagem é que o próprio barramento se torna um novo ponto único de falha: desloca o problema original sem o resolver. Adicionalmente, segue o princípio de *smart pipes, dumb endpoints*, concentrando lógica de integração num componente externo aos serviços, o que cria dependência forte no barramento e dificulta a evolução independente de cada serviço.

Microserviços com DDD (abordagem adotada). Cada domínio funcional é implementado como um serviço autónomo, com a sua própria base de dados e ciclo de deployment independente. As fronteiras entre serviços são definidas por **Bounded Contexts** do Domain-Driven Design: cada serviço modela apenas o conceito relevante dentro da sua fronteira, sem partilhar entidades com outros serviços. Por exemplo, Product no catalog-service contém nome, descrição e atributos; no cart-service existe apenas productId + quantity, sem entidade JPA partilhada, sem acoplamento estrutural. Esta

abordagem permite avaliar empiricamente os trade-offs de isolamento de falhas, latência inter-serviço e deployability que motivam a decomposição.

1.3 Questões de Investigação e hipóteses

O trabalho é orientado por três questões de investigação:

QI	Questão	Hipótese	Critério de validação
QI1	A decomposição em microserviços elimina ou apenas redistribui o acoplamento entre domínios?	H1: A decomposição substitui acoplamento estrutural compile-time (FKs JPA, entidades partilhadas, deploy conjunto) por acoplamento temporal runtime (REST síncrono) e acoplamento eventual (Kafka), sem reduzir o número total de dependências cross-domain.	Contar dependências antes e depois da migração: FKs cross-domain, imports JPA e entidades partilhadas (monólito) vs. Feign Clients, tópicos Kafka e propagação de identidade (microserviços). H1 é confirmada se: $total_micro \geq total_mono$.
QI2	A que custo em latência se paga a decomposição em fluxos de complexidade crescente?	H2: O overhead de latência p95 introduzido pela decomposição é desprezável em fluxos simples (T1: leitura, 1 hop) e dominante em fluxos transacionais coordenados (T3: checkout, 3 hops síncronos). O rácio $p95(micro)/p95(mono)$ é significativamente maior em T3 do que em T1.	Medir p95 de T1 e T3 em ambas as arquiteturas com k6 (10 VUs, 3 min). Calcular rácio $p95(micro)/p95(mono)$ por cenário. H2 confirmada se $rácio_T3 \gg rácio_T1$. Complementar com trace Jaeger do T3 nos microserviços para decompor o tempo por hop.
QI3	A comunicação assíncrona via Kafka melhora ou degrada a consistência sob falha parcial face à transação ACID do monólito?	H3: Sem mecanismos de compensação (Saga, idempotência e DLQ), a comunicação assíncrona apresenta maior potencial de inconsistência do que a transação ACID do monólito.	Executar cenário zombie (inventory-service parado durante checkout) e observar estado das BDs e tópico Kafka. Executar cenário equivalente no monólito (falha da BD a meio da transação) e verificar rollback automático. H3 confirmada se os microserviços apresentarem estados inconsistentes que não surgem no monólito.
QI4	Qual o custo operacional em repouso da decomposição face ao monólito?	H4: O overhead fixo de RAM em idle dos microserviços é significativamente superior ao monólito (estimado 5–8×), dominado pelo custo por processo JVM e pela infraestrutura de suporte (Kafka, Zookeeper, 6 instâncias PostgreSQL).	Medir RAM e CPU em idle com docker stats em ambas as arquiteturas (24 amostras × 5s, 3 corridas, mediana). Calcular rácio $total_micro / total_mono$. H4 confirmada se $rácio > 3\times$.

2 ENQUADRAMENTO TEÓRICO

A bibliografia consultada concentra-se em três áreas. A primeira diz respeito à decomposição arquitetural propriamente dita: Newman [2021] sistematiza as motivações, padrões e armadilhas da transição para microserviços; Fowler e Lewis [2014] cunham a definição operacional do termo; Richardson [2018] cataloga os padrões aplicáveis. A segunda área trata do desenho do domínio: Evans [2003] introduz os conceitos de Bounded Context e *Ubiquitous Language*, que orientam neste trabalho a definição das fronteiras entre serviços. A terceira aborda os padrões de resiliência e consistência distribuída: circuit breakers, Saga, *outbox*, idempotência relevantes para a compreensão das limitações que a decomposição introduz e dos mecanismos disponíveis para as mitigar.

Do ponto de vista metodológico, optou-se por uma abordagem comparativa entre o sistema original (AS-IS) e a proposta (TO-BE). A comparação é estabelecida em condições de paridade funcional, uma condição que exigiu intervenções no monólito original para o equiparar funcionalmente à proposta, e os resultados são interpretados à luz dos quatro atributos de qualidade identificados no enunciado: desempenho, disponibilidade, *deployability* e custo operacional.

3 SISTEMA DE REFERÊNCIA: AS-IS (MONÓLITO)

3.1 Arquitetura do Monólito XCommerce

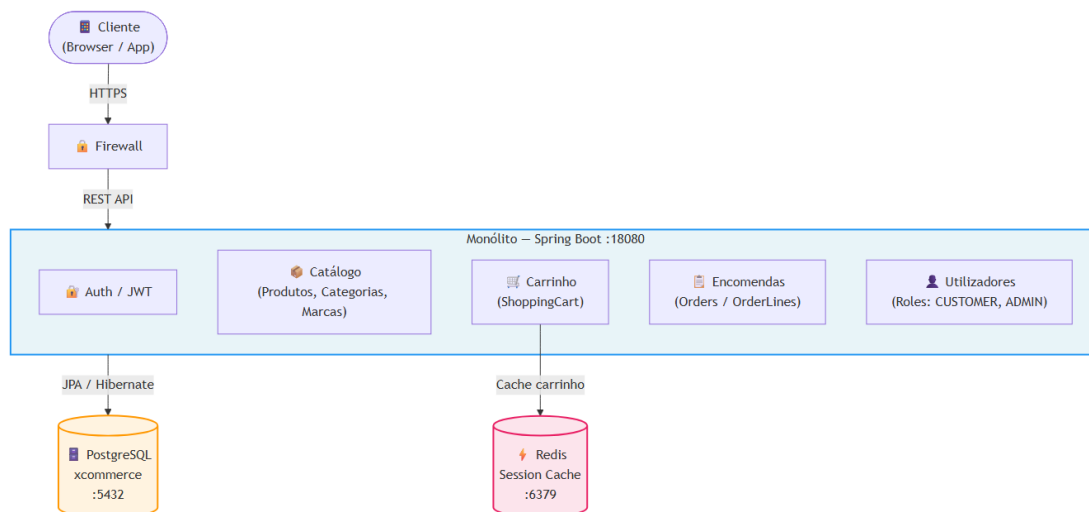


Figura 1, Arquitectura do sistema monolítico XCommerce

O XCommerce na sua forma original é uma aplicação Spring Boot com Java 8, organizada internamente em camadas (controllers, serviços, repositórios e entidades) mas implantada como um único artefacto. Toda a funcionalidade: autenticação, gestão do catálogo de produtos, carrinho de compras e processamento de encomendas, corre no mesmo processo, partilha o mesmo gestor de persistência JPA e escreve numa única instância PostgreSQL. O carrinho usa adicionalmente Redis como cache em memória.

Internamente, o código segue o padrão MVC clássico, com as responsabilidades distribuídas por cerca de três dezenas de classes agrupadas por camada. A separação lógica é clara, controladores que recebem requests, serviços que orquestram, repositórios que persistem, mas não corresponde a qualquer fronteira de deployment ou de falha. Todos os módulos partilham o mesmo *connection pool* JDBC e o mesmo classpath, pelo que uma falha originada num domínio propaga-se imediatamente aos restantes.

Internamente, o código organiza-se em 33 classes distribuídas por 5 packages:

Package	Conteúdo
controllers	UserRestController, OrderRestController, ShoppingCartRestController
services	UserService, OrderService, ShoppingCartService
data/entities	User, Order, OrderLine, Product, Brand, Category, Review, Authority, BaseEntity, OrderStatus, IllegalStatusChangeException
data/repositories	UserRepository, OrderRepository, ProductRepository, BrandRepository, CategoryRepository, ReviewRepository, AuthorityRepository, BaseRepository
security	JwtHelper, JwtInterceptingFilter, SecurityConfigurer, UserAuditor

Esta estrutura segue um padrão MVC-SOA: existe separação lógica entre camadas, mas todos os módulos partilham o mesmo EntityManager JPA e o mesmo schema de base de dados. Não existe qualquer fronteira de deployment ou de falha entre os domínios.

O acoplamento entre domínios é estrutural: OrderLine referencia diretamente a entidade JPA Product; o ShoppingCart armazena em cache Redis a entidade Product completa, incluindo preço, desconto, peso e relações com Brand e Category. Qualquer módulo pode aceder a dados de qualquer outro domínio sem restrição.

Do ponto de vista operacional, uma alteração em qualquer módulo (mesmo que mínima) obriga ao redeployment completo da aplicação. Não existe mecanismo de atualização independente por domínio.

3.2 Limitações que motivam a Decomposição

A primeira limitação estrutural é o **acoplamento entre os domínios do carrinho e do catálogo**. O carrinho não armazena meras referências aos produtos: guarda em cache Redis a entidade JPA completa, incluindo preço, peso, desconto, categoria e marca. Esta opção, eficaz em termos de leitura, transforma qualquer alteração ao modelo de produto numa potencial corrupção do estado do carrinho.

Acoplamento total no checkout. Quando um utilizador faz checkout, o sistema executa numa única operação: valida o utilizador, lê o carrinho, percorre os produtos e cria a encomenda. Uma falha em qualquer um destes passos (por exemplo,

sobrecarga no processamento de encomendas) derruba simultaneamente todas as funcionalidades da aplicação, incluindo operações completamente independentes como a consulta do catálogo.

Escalabilidade monolítica. O catálogo de produtos é tipicamente o domínio com maior volume de tráfego, enquanto o checkout é uma operação pontual. No monólito não é possível escalar um domínio de forma independente: aumentar a capacidade do catálogo implica replicar toda a aplicação.

A estas limitações estruturais somam-se duas limitações funcionais que importa explicitar, porque influenciam diretamente a comparação entre arquiteturas. A versão original do XCommerce **não valida o stock no checkout**, apesar de o campo correspondente existir na entidade **Product**; e os controladores **utilizam um identificador de utilizador fixo** em vez de extrair a identidade do token JWT. Estas duas limitações, sendo funcionais e não arquiteturais, foram corrigidas no decorrer deste trabalho para garantir paridade na comparação experimental

3.3 Dependências Estruturais (AS-IS)

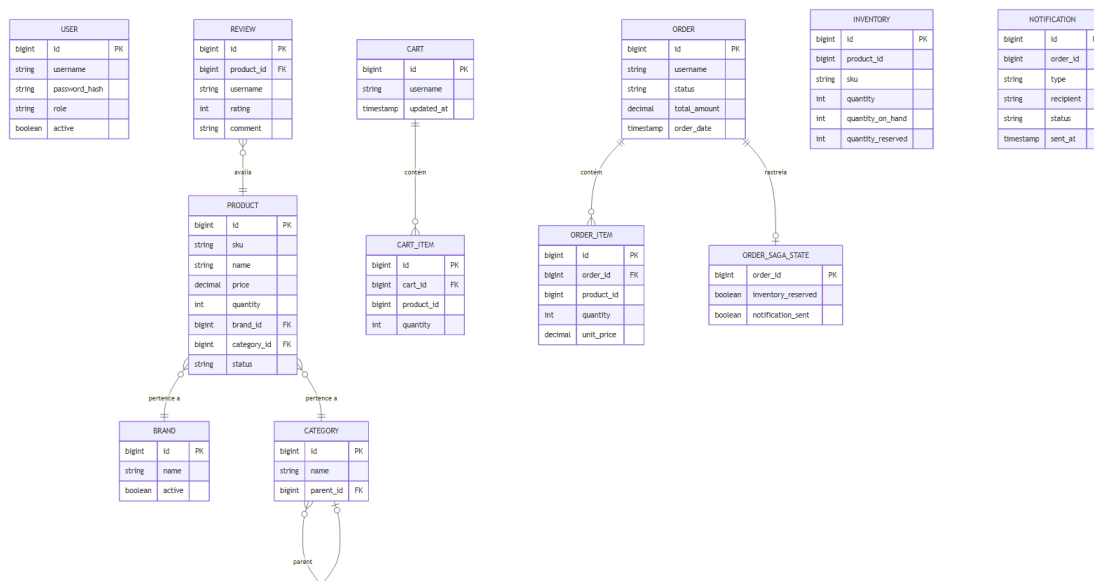


Figura 4, Modelo ER: entidades e relações

Dependência	Manifestação
Carrinho ↔ Catálogo	O carrinho guarda o objeto Product completo em cache (preço, peso, desconto, categoria e marca) em vez de guardar apenas o identificador do produto

Dependência	Manifestação
Encomenda ↔ Todos os domínios	O checkout acede em sequência ao utilizador, ao carrinho e aos produtos, criando a encomenda numa única operação sem isolamento entre domínios
Autenticação ↔ Domínio	A camada de segurança é transversal a todos os controladores; a integração com JWT ficou incompleta

4 ARQUITETURA TO-BE: MICROSERVIÇOS

4.1 Visão Geral da Decomposição

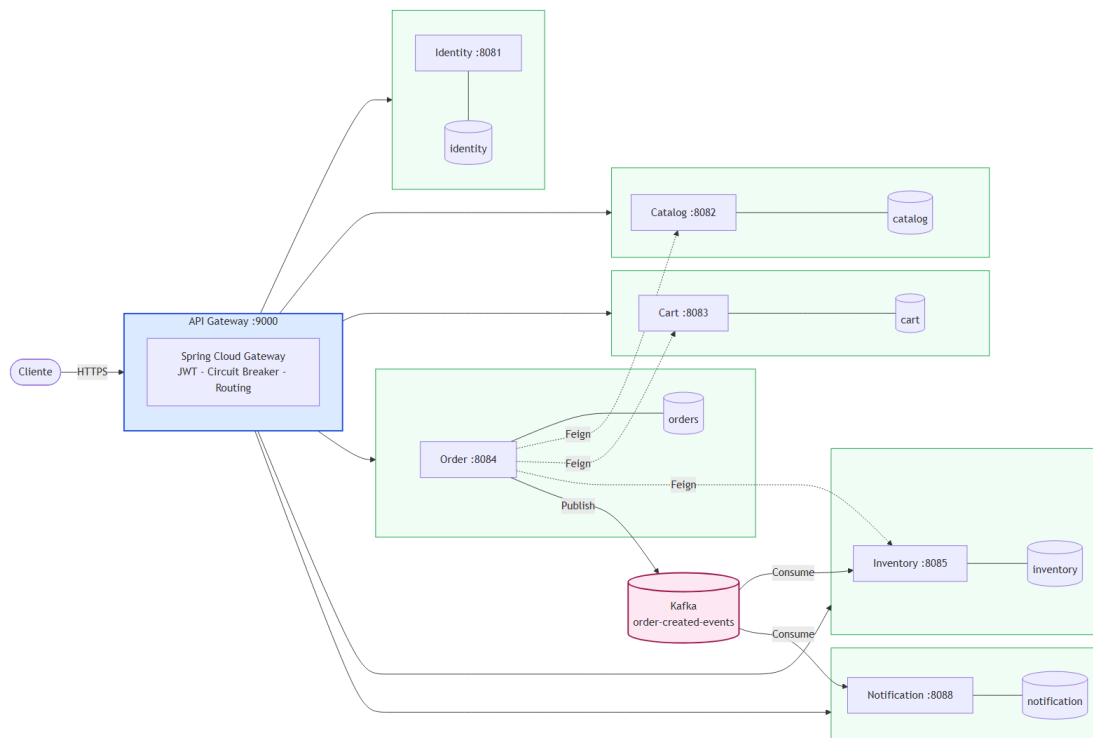


Figura 2, Arquitectura de microserviços XCommerce

A proposta decompõe o XCommerce em seis serviços de negócio autónomos e um *API Gateway*, alinhando cada serviço com um *Bounded Context* do Domain-Driven Design. A regra central é que cada serviço é o único proprietário dos dados do seu domínio: comunica com os restantes apenas através de APIs ou de mensagens assíncronas, nunca por acesso direto à base de dados alheia.

A escolha entre comunicação síncrona e assíncrona não é uniforme: depende, em cada caso, de uma questão concreta *o serviço que faz o pedido precisa da resposta para continuar?* Quando precisa (verificar stock antes de criar uma encomenda, obter o preço atualizado de um produto), opta-se por chamadas REST síncronas. Quando não precisa (notificar o utilizador, processar um pagamento de forma diferida), opta-se por mensagens publicadas em Kafka. Cada uma destas opções introduz problemas distintos, discutidos adiante.

4.2 Justificação da Decomposição:

Cart / Catalog

- **Problema que resolve:** No monólito, CartItem guarda entidade JPA Product completa, criando acoplamento estrutural. Cart e Catalog partilham o mesmo schema.
- **Decisão:** Cart passa a guardar apenas productId e quantity. Elimina-se o acoplamento estrutural.
- **Alternativa:** Manter Cart e Catalog no mesmo serviço (CatalogCart service mais agregado).
- **Trade-off introduzido:** Cart deixa de ter acesso direto a dados do produto (nome, preço). Para mostrar o carrinho com detalhes, order-service precisa de chamar catalog-service via REST (dependência em runtime). Se catalog-service falhar, o carrinho mostra apenas IDs.

Identity

- **Problema que resolve:** No monólito, autenticação e perfil de utilizador estão acoplados na mesma entidade. Qualquer evolução do mecanismo de autenticação (por exemplo, integração com um Identity Provider externo) implica tocar nos dados de perfil.
- **Decisão:** Um único identity-service gere credenciais, JWT e perfis dentro da mesma transação local.
- **Alternativa considerada:** Separar em dois serviços (auth e user) comunicando por sincronização cruzada. Rejeitada porque ambos modelariam o mesmo conceito de domínio (utilizador), sem ubiquitous language distinta, e a sincronização imediata em ambos os sentidos introduziria uma janela de inconsistência sem benefício de domínio.
- **Trade-off introduzido:** Centralizam-se duas responsabilidades num único serviço. Quando o sistema evoluir para um IdP externo, o identity-service permanece o ponto natural de integração, podendo nesse momento, e só nesse momento, separar-se o perfil se houver justificação de domínio.

Inventory (extensão funcional)

- **O que era no monólito:** campo quantity em Product, não validado no checkout
- **Decisão:** Criar inventory-service que valida e decrementa stock no checkout
- **Justificação:** Completar a semântica funcional era necessário para tornar o sistema comparável em condições reais
- **Trade-off introduzido:** A coordenação com o inventory-service é assíncrona via Kafka, não síncrona. O order-service publica order-created-events quando cria a encomenda em estado intermédio (HANDLING); o inventory-service consome o evento, valida e decrementa stock, e publica stock-reservation-events com o resultado; o order-service consome essa resposta e transita a encomenda para CONFIRMED ou CANCELLED. Esta opção desacopla temporalmente os dois serviços, mas introduz um estado intermédio observável: se o inventory-service estiver indisponível quando o evento é publicado, a encomenda permanece em HANDLING sem mecanismo automático de recuperação. Este é precisamente o cenário explorado pelo Workflow 4 (estado zombie).

Notification

- **Decisão:** Assíncrono via Kafka: notificações não bloqueiam resposta ao utilizador
- **Alternativa:** Chamada REST síncrona após checkout
- **Trade-off:** Se Kafka falhar, notificações perdem-se (sem dead-letter queue implementada).

4.3 Papel do API Gateway

Sem um ponto de entrada centralizado, o cliente precisaria de conhecer os endereços de **seis serviços** distintos, gerir o token JWT em cada chamada e lidar individualmente com falhas de cada serviço. O API Gateway resolve este problema ao ser o único componente com o qual o cliente comunica diretamente.

As suas quatro responsabilidades são:

1. Ponto único de entrada. Toda a comunicação externa entra pela porta 9000. O cliente nunca sabe que existem serviços separados por baixo: a estrutura interna da aplicação fica completamente encapsulada.

2. Routing por path. Cada pedido é encaminhado para o serviço correto com base no caminho do URL: `/products/**` vai para o `catalog-service`, `/rest/order/**` vai para o `order-service`, e assim por diante.

3. Autenticação e autorização. O Gateway valida a assinatura do token JWT em cada pedido, extrai o utilizador e o seu papel, e injeta os headers `X-User-Name` e `X-User-Role` antes de encaminhar o pedido. Os serviços internos confiam nesses headers sem voltar a validar o token: a responsabilidade de autenticação está centralizada num único ponto.

4. Resiliência por rota. Cada rota tem um circuit breaker com timeout de 5 segundos via Resilience4j. Quando um serviço demora a responder ou falha, o Gateway ativa um endpoint de fallback (`/fallback/*`) que devolve uma resposta degradada em vez de propagar o erro ao cliente.

O Gateway introduz, contudo, dois riscos. É um **ponto único de falha**: se ficar indisponível, toda a aplicação fica inacessível. Em produção este risco seria mitigado com múltiplas instâncias, mas no protótipo corre apenas uma. Acrescenta também **latência adicional** em cada pedido, uma vez que todos passam por mais uma camada antes de chegar ao serviço destino.

É importante notar que o circuit breaker protege contra cascatas de falhas, mas não resolve problemas de consistência de dados. Se o `order-service` aceitar um pedido e o `inventory-service` falhar a seguir, o circuit breaker não desfaz a encomenda já criada.

4.4 Introdução de Circuit breakers

A introdução de chamadas remotas obriga a planear para falhas. Todas as rotas do gateway são protegidas por **circuit breakers** (implementados com a biblioteca Resilience4j) que cortam pedidos a um serviço que esteja a falhar repetidamente, evitando cascatas e libertando recursos. As chamadas síncronas entre serviços (por exemplo, do serviço de encomendas para o de inventário) estão também protegidas por timeouts de cinco segundos. Uma falha do serviço de

catálogo é, por isso, contida ao seu domínio: as restantes operações continuam disponíveis, e os pedidos para o catálogo recebem uma resposta degradada em vez de bloquearem indefinidamente.

A comunicação assíncrona introduz, por seu turno, três limitações que importa explicitar. A entrega *at-least-once* característica do Kafka admite duplicação de mensagens, problema que se mitiga com idempotência ao nível do consumidor, não implementada neste protótipo. A ausência de coordenação transacional entre serviços admite estados intermédios em que algumas operações foram aplicadas e outras não, o caso paradigmático é o stock decrementado sem encomenda confirmada, mitigável com o padrão Saga e respetivas ações compensatórias, igualmente não implementado. Finalmente, mensagens cujo processamento falha repetidamente são perdidas, a menos que seja configurada uma *dead-letter queue*, o que também não foi feito. Estas três limitações são deliberadamente mantidas no protótipo, porque sustentam empiricamente a investigação à QI3.

4.5 Database per Service

A decomposição por domínios obriga ao isolamento dos dados. Se o order-service pudesse ler diretamente da base de dados do catalog-service, ficaria acoplado ao schema interno desse serviço, qualquer alteração ao catálogo poderia partir as encomendas, reproduzindo entre serviços o mesmo acoplamento que existia no monólito. O padrão database per service resolve isto com uma regra única: cada serviço é o dono exclusivo dos seus dados e a comunicação entre serviços faz-se por API, nunca pela base de dados.

Existiam três abordagens possíveis. Uma única instância PostgreSQL com bases de dados lógicas separadas seria a mais simples de operar, mas mantém um single point of failure: se a instância cai, todos os serviços ficam indisponíveis em simultâneo, o que contradiz a razão de ter decomposto o sistema. Misturar tecnologias por domínio (Redis para o carrinho, PostgreSQL para os restantes) seria a versão mais fiel ao padrão em produção, mas introduzia complexidade de operação fora do âmbito do trabalho. Optámos pela terceira: uma instância PostgreSQL dedicada por serviço, todas com a mesma tecnologia para manter o ambiente homogéneo, mas com isolamento físico real entre processos.

A implementação tem **seis instâncias PostgreSQL independentes** (db-identity, db-catalog, db-cart, db-orders, db-inventory, db-notification), cada uma no seu contentor. Cada serviço aponta apenas para a sua, não existem foreign keys cruzadas, e o ciclo de vida de cada base de dados (arranque, paragem, backup) é independente dos restantes.

O que esta abordagem não resolve são as garantias transacionais entre serviços: não existe rollback automático se o stock for decrementado e o pagamento falhar a seguir. É exatamente esta limitação que justifica o uso do padrão Saga descrito mais à frente, não como sofisticação opcional, mas como requisito direto da decomposição que isola os dados.

4.6 Comunicação Inter-Serviço: Kafka vs REST

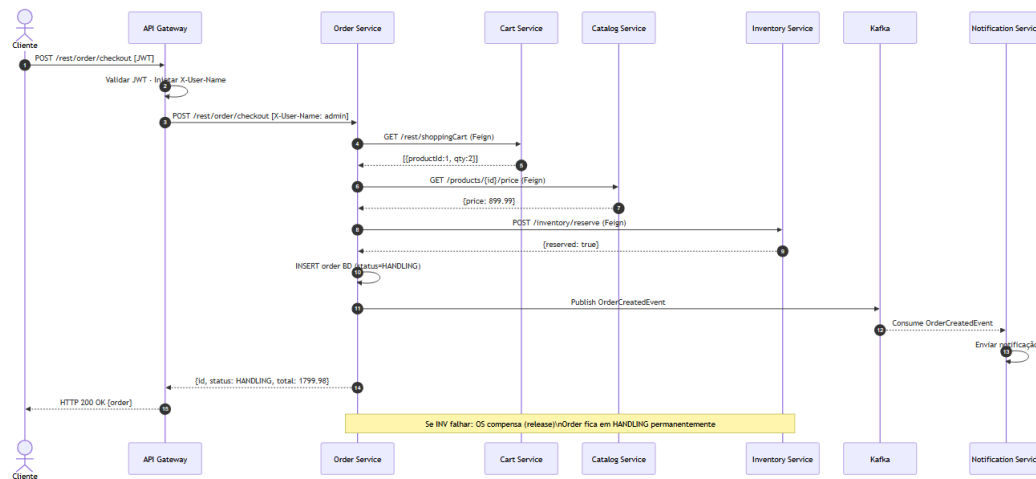


Figura 3, Diagrama de sequência do fluxo de checkout

A escolha entre comunicação síncrona e assíncrona não foi uniforme: dependeu de uma questão concreta para cada caso: o serviço que faz o pedido precisa da resposta para continuar?

REST síncrono foi utilizado nas situações em que a resposta é necessária para prosseguir o fluxo. Durante o checkout, o order-service precisa de saber se há stock disponível antes de criar a encomenda: sem essa resposta, não pode avançar. Da mesma forma, precisa de obter o preço atual do produto a partir do catalog-service e os itens do carrinho a partir do cart-service.

O trade-off é direto: cada chamada REST adiciona latência ao checkout, e se qualquer um destes serviços estiver indisponível, o checkout falha. É uma dependência em runtime que não existia no monólito, onde tudo corria no mesmo processo. O inventory-service consome order-created-events, valida o stock disponível e tenta decrementá-lo numa transação local. Publica em stock-reservation-events o resultado: STOCK_RESERVED_SUCCESS ou STOCK_RESERVATION_FAILED.

Kafka assíncrono foi utilizado nas situações em que a resposta não é necessária para continuar. Após a encomenda ser criada, o order-service publica os eventos e a resposta ao utilizador não precisa de esperar pelo processamento do pagamento nem pelo envio de notificações. O fluxo completo é o seguinte:

1. Quando o utilizador faz checkout, o order-service persiste a encomenda em estado intermédio (HANDLING) e publica em order-created-events o identificador da encomenda e os itens.
2. O inventory-service consome order-created-events, valida o stock disponível e tenta decrementá-lo numa transação local. Publica em stock-reservation-events o resultado: STOCK_RESERVED_SUCCESS ou STOCK_RESERVATION_FAILED

3. O `order-service` consome `stock-reservation-events`. Em caso de sucesso, transita a encomenda para `CONFIRMED` e publica em `order-confirmed-events`. Em caso de falha, transita para `CANCELLED` e publica em `order-cancelled-events`.
4. O `notification-service` consome ambos os tópicos (`order-confirmed-events` e `order-cancelled-events`) para notificar o utilizador do desfecho.

Esta abordagem introduz três limitações que não foram resolvidas e que são assumidas explicitamente.

Entrega duplicada. O Kafka garante entrega *at-least-once*: em caso de falha ou reprocessamento, a mesma mensagem pode ser entregue mais do que uma vez. A solução seria implementar idempotência: cada mensagem teria um identificador único e o serviço verificaria se já a processou antes de agir. Não está implementado.

Ausência de compensação. O checkout decorre em dois momentos distintos: o stock é decrementado por REST síncrono antes de a encomenda ser criada, e o pagamento é processado depois de forma assíncrona. Se o pagamento falhar, a encomenda fica cancelada mas o stock não é repostado: ficou decrementado sem que nenhuma venda tenha sido concluída. A solução seria implementar o padrão **Saga**: uma sequência de transações locais coordenadas, onde cada passo tem uma ação compensatória definida. Se o pagamento falhar, a Saga dispararia automaticamente a reposição do stock. Não está implementado.

Mensagens perdidas. Se o `notification-service` falhar ao processar uma mensagem (por exemplo, por um erro interno ou por estarem temporariamente indisponíveis) essa mensagem perde-se sem hipótese de recuperação. A solução seria configurar uma **dead-letter queue**: mensagens que falham após N tentativas seriam encaminhadas para uma fila separada, onde poderiam ser inspecionadas e reprocessadas manualmente. Não está configurado em nenhum serviço.

5 ANÁLISE DE WORKFLOWS

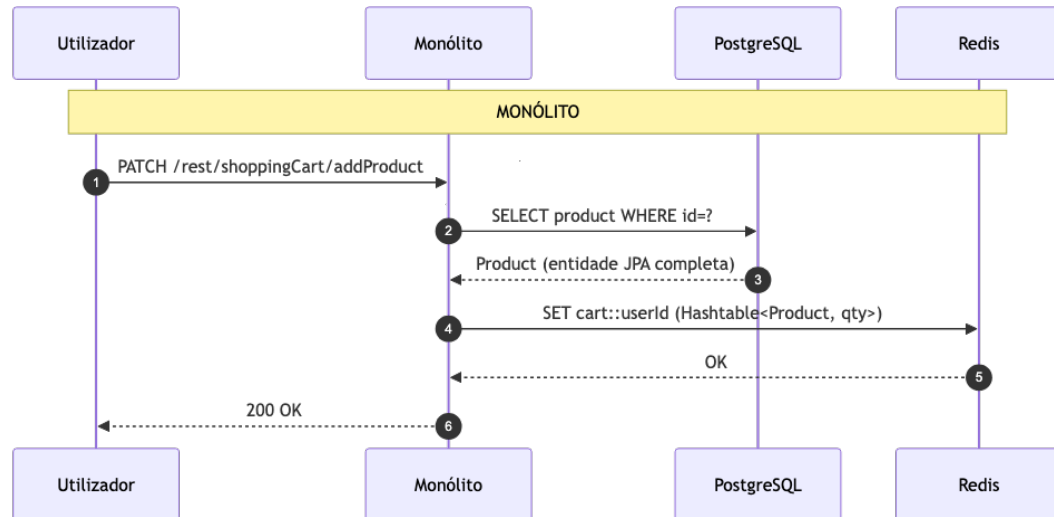
A análise comparativa entre as duas arquiteturas concentra-se em quatro fluxos representativos. Os três primeiros são de caminho-feliz e cobrem o espectro de complexidade do sistema: uma operação simples (Workflow 1), uma operação de leitura agregada (Workflow 2) e uma operação transacional complexa (Workflow 3). O quarto é deliberadamente um caminho de falha, a injeção controlada de uma falha a meio do checkout, para evidenciar uma propriedade do sistema que só é observável sob falha parcial. A escolha destes quatro casos cobre as dimensões relevantes para as três questões de investigação.

Cada workflow é apresentado segundo a mesma estrutura: o fluxo nas duas arquiteturas é representado num diagrama de atividade lado a lado; depois identifica-se o proprietário de cada dado, o que é lido e escrito, as dependências runtime, as falhas parciais possíveis, o estado inconsistente que pode resultar, o mecanismo de mitigação adotado e a lacuna que permanece.

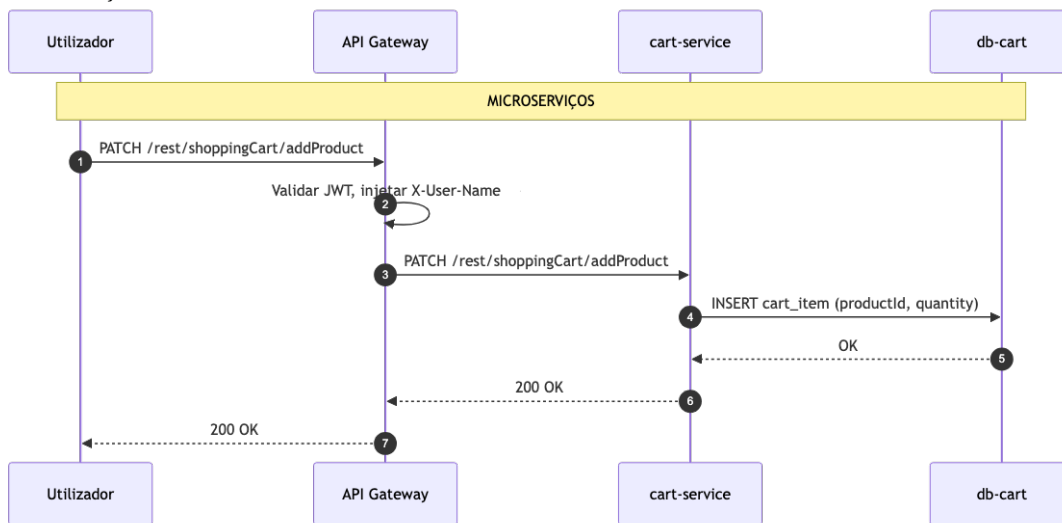
5.1 Workflow 1: Adicionar Produto ao Carrinho

Este é o fluxo mais simples do sistema e o caso mais limpo para ilustrar a redistribuição de acoplamento prevista por H1. No monólito, o serviço de carrinho acede diretamente ao repositório de produtos, carrega a entidade JPA completa e armazena-a em cache Redis. Há acoplamento estrutural: o modelo do produto vive dentro do estado do carrinho. Nos microserviços, a fronteira entre carrinho e catálogo é explícita: o carrinho mantém apenas o identificador do produto e a quantidade.

Monolítico:



Microserviços:



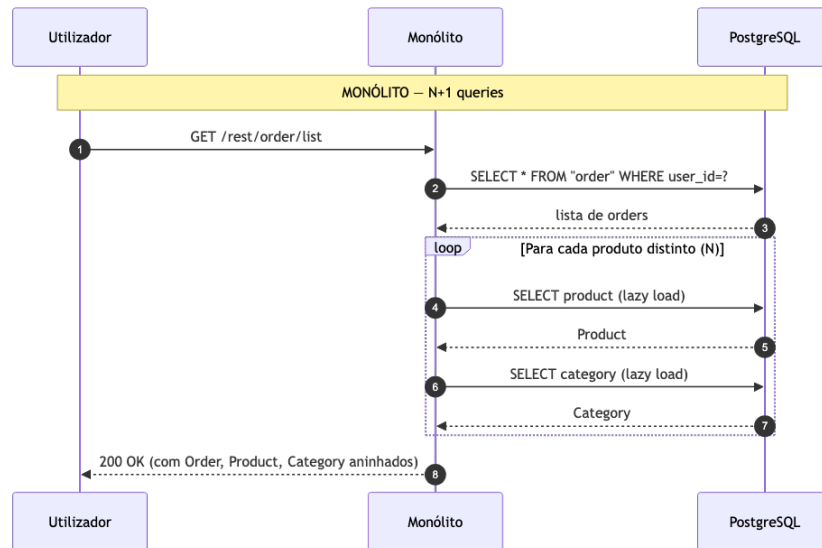
Aspeto	Monólito	Microserviços
Proprietário dos dados	Schema único (product e cart partilham BD)	cart-service e catalog-service em BDs distintas
Operações	1 SELECT na entidade Product + 1 SET em Redis	1 INSERT local no cart-service
Acoplamento	Estrutural: cart guarda entidade JPA Product completa	Apenas productId e quantity
Modo de falha	Falha do PostgreSQL afeta todos os domínios	Cada serviço falha isoladamente
Estado inconsistente possível	Não (transação local atómica)	cartItem com productId de produto inexistente, se o catálogo for alterado depois
Mitigação implementada	n/a	Circuit breaker no gateway; cart aceita sem validação cruzada
Lacuna assumida	n/a	Sem validação de existência do produto em runtime no cart-service

Este workflow é a primeira instância concreta do que H1 prevê: o acoplamento estrutural do monólito (entidade JPA partilhada em cache) é substituído por uma referência simples por identificador, à custa de uma chamada de rede futura sempre que se precisar de informação do produto.

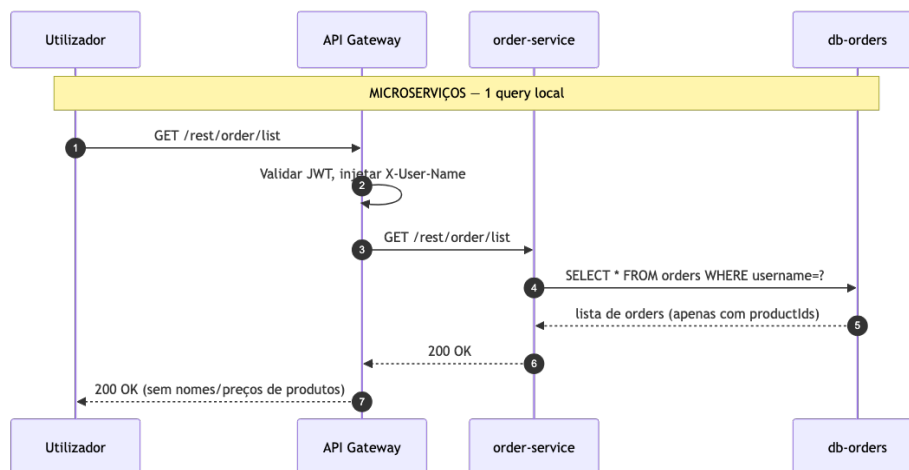
5.2 Workflow 2: Consultar Encomenda

Este fluxo expõe o problema **N+1 queries** do monólito (que deriva do mapeamento JPA com *lazy loading*) e mostra como a decomposição o transforma, não o elimina, redistribui-o. No monólito, listar as encomendas de um utilizador desencadeia uma consulta inicial seguida de duas consultas adicionais por cada produto distinto na encomenda. Para uma encomenda com 4 produtos, o motor JPA emite 11 consultas SQL: $3 + 2N$. Nos microserviços, o serviço de encomendas devolve apenas os identificadores dos produtos; cliente que queira detalhes consulta o catálogo separadamente.

Monolítico:



Microserviços:



Aspeto	Monólito	Microserviços
Proprietário dos dados	Único schema com JOINS possíveis	order-service é dono apenas de xcommerce_orders
Consultas SQL	3 + 2N (problema N+1)	1 consulta local
Resposta	Rica: order + produtos + categorias aninhadas	Pobre: order + lista de productId
Modo de falha	Falha da BD afeta toda a aplicação	Falha do catálogo não impede a listagem de encomendas
Trade-off	Resposta completa numa só chamada, mas com custo SQL desproporcional	Resposta económica, mas obriga o cliente a chamar catálogo separadamente para detalhes
Mitigação implementada	n/a	Cliente faz chamadas adicionais ao catálogo se precisar de detalhes
Lacuna assumida	Custo SQL não otimizado (sem JOIN explícito)	Sem composição de dados no gateway

Este workflow ilustra que a decomposição não é gratuita: o monólito serve “tudo numa chamada” mas paga em queries SQL; os microserviços servem rapidamente uma resposta parcial, deslocando o custo para o cliente ou para chamadas posteriores. A escolha depende do que se valoriza mais: **riqueza da resposta ou autonomia dos serviços**.

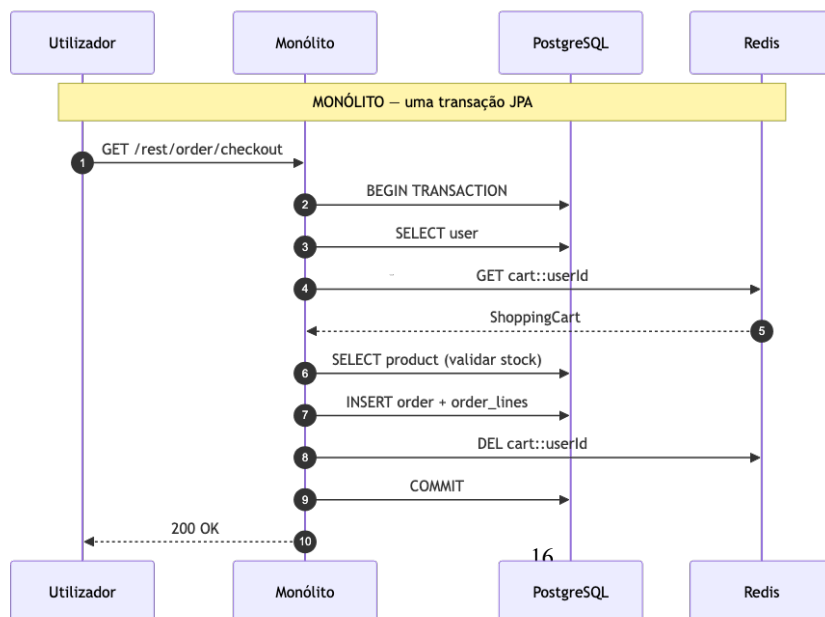
5.3 Workflow 3: Checkout

Este é o **fluxo central do trabalho**: a operação que toca todos os domínios e que sustenta empiricamente as hipóteses H2a (super-linearidade da latência) e H3 (consistência sob falha).

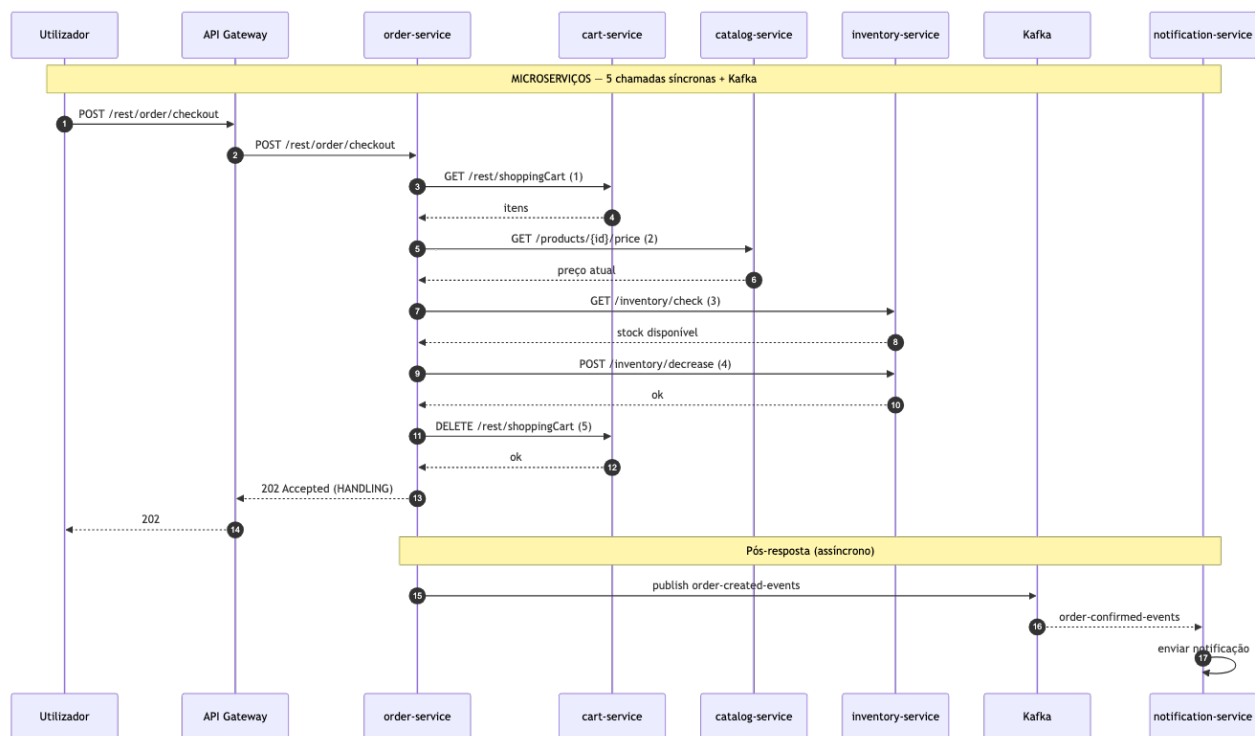
No monólito, o checkout executa numa única transação JPA: **lê o utilizador, valida stock, calcula totais, persiste a encomenda, esvazia o carrinho**. Tudo dentro do mesmo processo, com garantia ACID.

Nos microserviços, a mesma operação lógica é decomposta em duas fases: uma fase síncrona em que o order-service coordena três chamadas REST (cart-service para ler o carrinho, catalog-service para obter o preço atual de cada item, cart-service novamente para esvaziar o carrinho) e persiste a encomenda em estado intermédio; e uma fase assíncrona em que a confirmação de stock é orquestrada por troca de mensagens Kafka com o inventory-service, transitando a encomenda para CONFIRMED ou CANCELLED.

Monolito:



Microserviços:



Aspeto	Monólito	Microserviços
Proprietário dos dados	Schema único	5 BDs distintas, uma por serviço envolvido
Coordenação	1 transação JPA local	5 chamadas REST síncronas em série + 1 publicação Kafka
Hops síncronos	1 (BD local)	5 (cart, catalog, inventory ×2, cart)
Atomicidade	Garantida pelo motor JPA (ACID)	Sem garantia, operações persistem independentemente

Aspeto	Monólito	Microserviços
Modo de falha	Falha na BD → rollback automático completo	Falha em qualquer serviço → estado parcial intermédio
Mitigação implementada	n/a (rollback nativo)	Circuit breakers e timeouts no gateway e nos clientes Feign
Lacuna assumida	Sem validação assíncrona (notificações, eventos)	Sem Saga compensatória, sem idempotência, sem dead-letter queue

A cascata de cinco chamadas síncronas é a evidência visual de **H2a**: o que no monólito é uma única transação local torna-se uma sequência de cinco operações remotas, com o custo super-linear que se observa empiricamente abaixo.

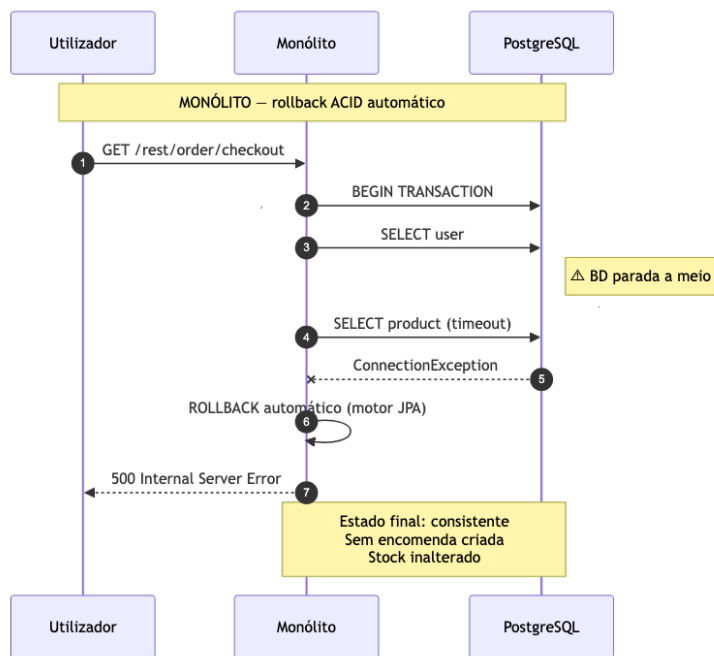
O facto de operações intermédias (decremento de stock, persistência da encomenda) já terem sido aplicadas quando a coordenação falha é exatamente o cenário que o Workflow 4 explora.

5.4 Workflow 4: Falha parcial

Este workflow não é caminho feliz: é a injeção deliberada de uma falha a meio do checkout para observar o estado dos dados. **Sustenta H3**, a hipótese de que, sem padrões de compensação implementados (Saga, idempotência, dead-letter queue), a comunicação assíncrona regride a consistência face à transação ACID do monólito.

Cenário. Nos microserviços, o inventory-service é parado deliberadamente antes do checkout. Quando o checkout é executado, a encomenda é persistida em estado intermédio (HANDLING) e o evento order-created-events é publicado em Kafka, mas o consumidor que deveria validar e decrementar stock não está disponível. No monólito, o cenário equivalente é parar a base de dados a meio da transação JPA do checkout.

Monolítico:



Microserviços:

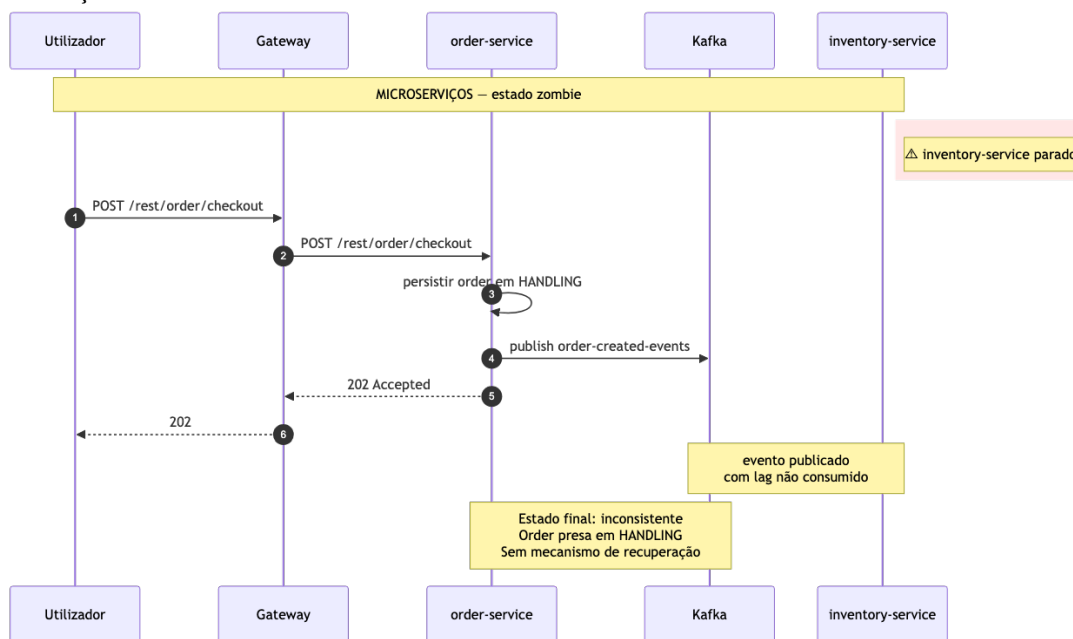


Tabela / Tópico	Estado esperado (sem falha)	Estado observado
xcommerce_orders.orders	CONFIRMED após processamento Kafka	HANDLING (encomenda zombie)
xcommerce_inventory.inventory	Decrementado pelo consumer Kafka	Sem alteração (consumer parado)
Tópico order-created-events	Consumido pelo inventory-group	Lag = 1, consumer group sem membros ativos
Carrinho	Limpo (último passo síncrono)	Limpo, irreversível

Lacunas evidenciadas nos microserviços.

- Sem Saga compensatória: o decremento que deveria ocorrer não ocorre, mas se tivesse ocorrido (cenário simétrico) não seria revertido em caso de falha posterior.
- Sem timeout/dead-letter queue no order-service que detete eventos não consumidos além de um determinado prazo.
- Sem operação manual ou automática de reconciliação implementada.

Este workflow é a evidência empírica direta de H3: no monólito, o motor JPA garante atomicidade gratuitamente; nos microserviços, essa garantia tem de ser reconstruída com padrões adicionais (Saga, idempotência, dead-letter queue): nenhum dos quais está implementado neste protótipo, e cuja ausência é deliberadamente assumida como limitação.

6 AVALIAÇÃO EXPERIMENTAL & RESULTADOS

6.1 Metodologia

Os testes foram executados em ambiente local (Apple M4 Pro, 48 GB RAM, macOS Darwin 25.4.0), com ambas as arquiteturas em contentores Docker isolados. As duas arquiteturas nunca correram em simultâneo para evitar interferência de recursos.

A ferramenta de carga utilizada foi o **k6** com 10 VUs constantes, janela de medição de 3 minutos por cenário e 2 runs por combinação — reporta-se a mediana dos p95. Os testes correm sem think time entre iterações, opção deliberada para saturar a arquitetura e medir o comportamento sob pressão sustentada.

As métricas de latência foram complementadas com **Jaeger** (traces distribuídos) para decompor o tempo por hop individual, e **Prometheus + Grafana** para monitorização em tempo real. O consumo de recursos foi medido com **docker stats** (24 amostras × 5s, 3 runs, mediana) e confirmado com métricas JVM via Actuator/Prometheus.

O dataset de teste foi idêntico nas duas arquiteturas: 50 utilizadores, 20 produtos, 250 encomendas pré-criadas (5 × 50 users, 4 itens cada).

6.2 Avaliação de H1 -Redistribuição do acoplamento

Objetivo da Experiência. Validar se a transição para microserviços reduz efetivamente a interdependência entre domínios funcionais ou se apenas altera a natureza do acoplamento. A hipótese H1 postula que a decomposição elimina o acoplamento estrutural (visível ao compilador) mas introduz acoplamento dinâmico (visível apenas em runtime) em volume igual ou superior, confirmando que a modularidade física não equivale necessariamente a independência funcional.

Método de Análise. Foi realizada uma auditoria estática comparativa entre as duas arquiteturas:

No Monólito: Contabilizaram-se relações JPA @ManyToOne diretas, injeções de repositórios de outros domínios em serviços centrais (ex: OrderService acedendo a ProductRepository) e a partilha de entidades de persistência em DTOs.

Nos Microserviços: Mapearam-se as dependências de runtime através de interfaces OpenFeign (REST), produtores/consumidores de tópicos Kafka e a dependência transversal de identidade via cabeçalhos HTTP (X-User-Name).

Resultados Consolidados. A tabela abaixo apresenta a métrica de "Pontos de Contacto entre Domínios" (Cross-Domain Coupling Points):

Categoria de Acoplamento	Indicador Técnico	Monólito	Microserviços
Estático (Compile-time)	Relacionamentos JPA e Foreign Keys	3	0
	Acesso Direto a Repositórios/Serviços	3	0
	Entidades JPA em DTOs (Leaking)	1	0
Dinâmico (Runtime)	Chamadas REST Síncronas (Feign)	0	3
	Tópicos Kafka (Eventos Cross-service)	0	4
	Propagação de Identidade (Headers)	0	6
Total	Pontos de Acoplamento	7	13

Análise e Interpretação dos Dados.

Os dados recolhidos confirmam a hipótese H1. Embora o acoplamento estático tenha sido reduzido a zero, comprovado pela ausência de imports cruzados e pela autonomia das bases de dados (PostgreSQL independente por serviço), o acoplamento total do sistema aumentou aproximadamente 85%.

Erosão das Barreiras Transacionais: No monólito, o acoplamento entre Order e Product era gerido pelo motor JPA com garantias ACID. Nos microserviços, este vínculo foi substituído pelo InventoryClient (REST) e pelo tópico order-created-events (Kafka). Isto transfere a responsabilidade da integridade do compilador para a lógica de aplicação (necessidade de padrões Saga e Idempotência).

Acoplamento Temporal: A dependência do order-service em relação aos serviços de cart, catalog e inventory via chamadas síncronas introduz um risco de disponibilidade em cascata que não existia na versão original.

Dependência de Infraestrutura: A introdução de 6 pontos de dependência de identidade (propagada pelo API Gateway) demonstra que a autonomia dos serviços é parcial; todos dependem agora de um contrato de cabeçalhos comum para funcionar.

Conclusão.

A decomposição não simplificou a rede de dependências; pelo contrário, tornou-a mais complexa ao torná-la invisível às ferramentas de análise estática tradicionais. O sistema trocou o "rigor do compilador" pela "fragilidade da rede", validando integralmente a hipótese de redistribuição de acoplamento.

6.3 Avaliação H2: Custo de Latência

Objetivo da Experiência. Determinar se o custo de latência introduzido pela decomposição cresce de forma proporcional ao número de chamadas síncronas em série ou de forma desproporcional. A hipótese H2 prevê que o overhead seja negligenciável em fluxos simples com um único salto e dominante em fluxos transacionais que coordenam múltiplos serviços.

Método. Foram definidos dois cenários de complexidade distinta.

O **cenário T1** representa uma operação de leitura leve (GET /rest/shoppingCart/get no monólito, GET /products nos microserviços), com um único salto efectivo em ambas as arquiteturas.

O **cenário T3** representa o fluxo de checkout completo (GET /rest/order/checkout no monólito, executado numa única transação JPA, e POST /rest/order/checkout nos microserviços, que coordena 3 chamadas REST síncronas em série, uma escrita local e uma publicação assíncrona no Kafka).

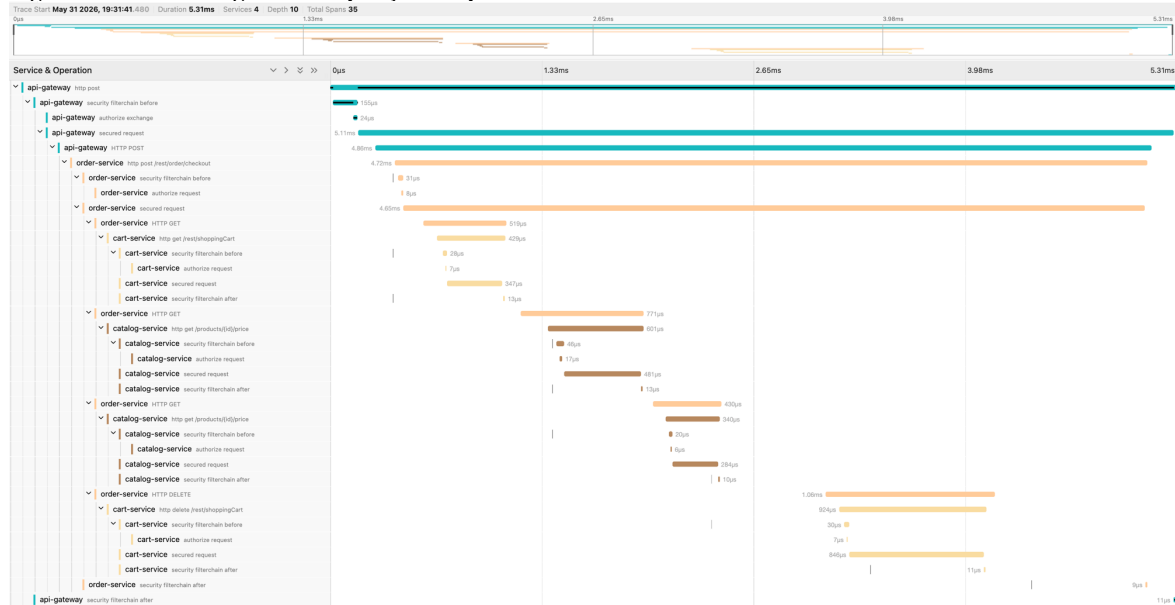
Cenário	Hops mono	Hops micro	Mono p95 (ms)	Micro p95 (ms)	Variação
T1 — leitura leve	1	1	8.9	13.7	+54%
T3 — checkout	1 (JPA)	3 REST + BD + Kafka	12.0	17.0	+42%

Cenário	Mono req/s	Micro req/s	Erros mono	Erros micro
T1	93	91	0%	0%
T3	58	57	0%	0%

Figura 1 — Dashboard H2a: latência e throughput (k6, 10 VUs)



Figura 2 — Trace Jaeger: decomposição temporal do checkout

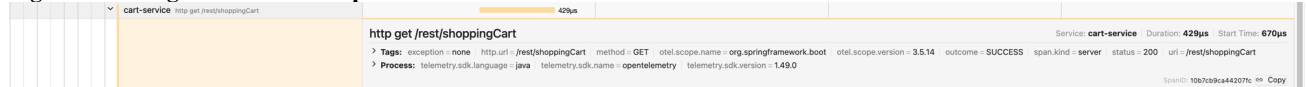


Trace POST /rest/order/checkout — duração total 5.31ms, 4 serviços, 35 spans.

Hops síncronos: cart-service (leitura) 519µs → catalog-service (preço produto 1) 771µs → catalog-service (preço produto 2) 430µs → cart-service (limpeza) 1.06ms.

Total rede: ~2.78ms = 59% do tempo do order-service.

Figura 3 — Jaeger: detalhe de 1 span



Span cart-service http get /rest/shoppingCart — Duration: 429µs, status: 200. Custo de 1 hop REST individual em localhost.

Análise e Interpretação dos Dados. Para uma operação simples (T1), a decomposição introduz um overhead de +4,8 ms em termos absolutos (+54%), atribuível essencialmente ao custo fixo de atravessamento do API Gateway, validação do token JWT, routing e injeção de cabeçalhos de identidade. Para uma operação complexa (T3), o overhead absoluto é de +5,0 ms (+42%), valor praticamente idêntico ao de T1, o que contraria a previsão de crescimento proporcional ao número de saltos.

A explicação reside no facto de o custo marginal de cada salto REST adicional, em rede local, ser da ordem de 1 ms — insuficiente para alterar significativamente o total, que continua a ser dominado pelo custo fixo do gateway. O trace Jaeger confirma esta leitura: 59% do tempo de processamento do order-service é já consumido em chamadas de rede em localhost. Esta observação encerra, contudo, uma ressalva crítica: em ambiente de produção com latências de rede reais de 10 a 50 ms por salto, os 3 saltos REST síncronos do checkout representariam entre 30 e 150 ms de overhead adicional. O ambiente

localhost subestima sistematicamente o custo da decomposição, e o sinal de alarme, 59% do tempo já consumido em rede mesmo em condições laboratoriais, está empiricamente presente.

Conclusão. A hipótese H2 é confirmada parcialmente. Em ambiente localhost, o overhead é dominado pelo custo fixo de gateway e não cresce com o número de saltos. A hipótese de crescimento super-linear é, contudo, consistente com o comportamento esperado em produção.

6.4 Avaliação H3: Consistência sob Falha Parcial

Objetivo da Experiência. Verificar se a comunicação assíncrona via Kafka, na ausência de padrões de compensação implementados (Saga, idempotência, dead-letter queue), apresenta maior potencial de inconsistência do que a transação ACID do monólito. A hipótese H3 prevê que uma falha de um consumidor a meio de um fluxo deixa o sistema num estado intermédio sem mecanismo automático de recuperação, ao passo que uma falha equivalente no monólito é absorvida pelo rollback automático do motor transaccional.

Método. Procedeu-se à injeção controlada de uma falha em cada arquitectura.

Nos microserviços, o inventory-service foi deliberadamente interrompido antes da execução de um checkout, de modo a que o evento order-created-events fosse publicado no Kafka mas o consumidor responsável pelo seu processamento não estivesse disponível.

No monólito, simulou-se uma condição de falha equivalente através da tentativa de checkout com um produto de stock nulo, provocando uma excepção de validação no interior da transação JPA. Em ambos os casos, registou-se o estado dos dados após a falha.

Resultados

Aspeto observado	Microserviços	Monólito
Falha provocada	Stop do serviço de inventário	Stop da base de dados
Resposta HTTP recebida pelo cliente	202 Accepted (sucesso aparente)	500 Internal Server Error
Estado da encomenda na base de dados	Persistida em estado intermédio (HANDLING)	Não criada (rollback)
Stock decrementado	Não (consumidor parado)	Não (rollback)
Mensagem Kafka pendente	Sim, com lag não nulo, sem consumidor ativo	Não aplicável
Mecanismo de recuperação automática	Nenhum	Rollback ACID automático
Estado final do sistema	Inconsistente, encomenda presa	Consistente

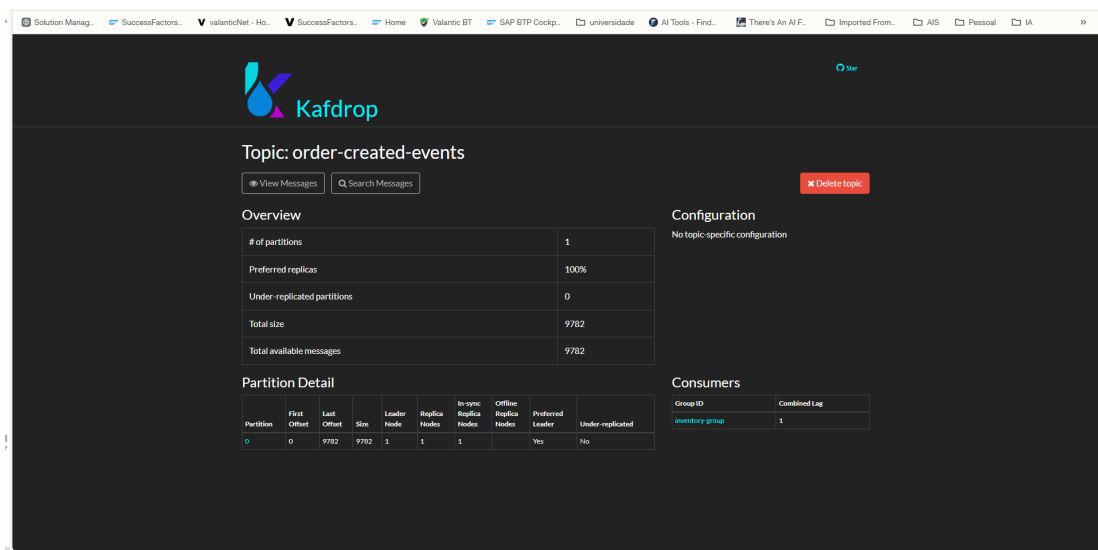


Figura 7, Tópico order-created-events com Combined Lag=1

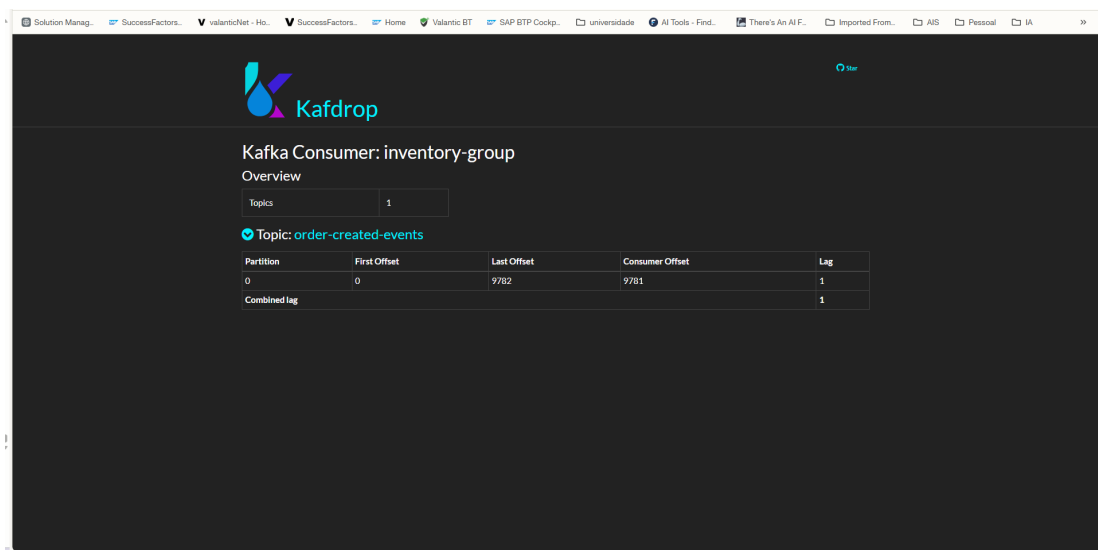


Figura 8, Consumer group inventory-group sem membros ativos

```
PS C:\Users\VALANTIC\OneDrive - valantic Business Technology\Desktop\ASID-XCommerce-Refactoring> docker exec xcommerce-d
b-orders psql -U postgres -d xcommerce_orders -c "SELECT id, username, status FROM orders ORDER BY id DESC LIMIT 5;"
 id | username | status
-----+-----+-----
 9780 | admin    | HANDLING
 9779 | admin    | CONFIRMED
 9778 | admin    | CONFIRMED
 9777 | admin    | CONFIRMED
 9776 | admin    | CONFIRMED
(5 rows)
```

Figura 9, Encomenda em estado HANDLING permanente (estado zombie)

```
I769175@G2Y6THK3MW ASID-XCommerce-Refactoring-1 % PGPASSWORD=postgres psql -h localhost -p 15432 -U postgres -d xcommerce \
-c "SELECT count(*) as total_encomendas FROM \"order\";"
total_encomendas
-----
          251
(1 row)

I769175@G2Y6THK3MW ASID-XCommerce-Refactoring-1 % PGPASSWORD=postgres psql -h localhost -p 15432 -U postgres -d xcommerce \
-c "SELECT id, name, quantity FROM product WHERE id IN (1, 2375);"
 id | name | quantity
----+-----+-----
  1 | S10  |         5
2375 | Produto Sem Stock |         0
(2 rows)

I769175@G2Y6THK3MW ASID-XCommerce-Refactoring-1 % PGPASSWORD=postgres psql -h localhost -p 15432 -U postgres -d xcommerce \
-c "SELECT status FROM \"order\" WHERE id=(SELECT MAX(id) FROM \"order\");"
status
-----
      0
(1 row)
```

Figura 10 — Contraste: rollback automático no monólito

Evidência experimental. A Figura 7 mostra o tópico order-created-events no Kafdrop imediatamente após a paragem do inventory-service: o Last Offset é 9782 e o Consumer Offset é 9781, resultando num Combined Lag de 1, a mensagem foi publicada pelo Order Service mas nunca consumida. A Figura 8 confirma pela perspetiva do consumer group inventory-group: ausência de membros ativos e lag=1 persistente. A Figura 9 mostra o estado da base de dados do Order Service: a encomenda com id=9780 permanece em estado HANDLING indefinidamente, enquanto todas as encomendas anteriores apresentam estado CONFIRMED. Este resultado contrasta diretamente com o monólito: quando a base de dados foi interrompida a meio da transação JPA, o sistema retornou HTTP 500 e nenhuma encomenda foi criada, o sistema regressou automaticamente ao estado anterior.

Análise do Estado Zombie. O estado zombie resulta da combinação de três características arquiteturais: (1) o Order Service opera de forma otimista, cria a encomenda localmente antes de confirmar que os serviços dependentes estão disponíveis; (2) a publicação no Kafka é fire-and-forget, o Order Service considera o seu trabalho concluído após o publish, independentemente de haver consumidores ativos; (3) não existe nenhum mecanismo de timeout ou compensação que reverta o estado da encomenda caso o inventory-service não responda. O resultado é que a encomenda fica presa em HANDLING, nem confirmada, nem cancelada, nem revertida automaticamente.

Interpretação. A diferença de comportamento entre as duas arquiteturas é qualitativa, não meramente quantitativa. No monólito, a transação JPA é atômica: ou todas as operações se aplicam, ou nenhuma se aplica. Quando a base de dados falha, o sistema reverte automaticamente ao estado anterior, e o cliente recebe uma resposta de erro clara que pode ser interpretada como uma indicação para tentar novamente. O sistema regressa, sem intervenção, a um estado consistente.

Nos microserviços, a operação foi aceite pelo gateway com resposta 202, o cliente é informado de sucesso aparente, ainda que parcial, e a encomenda foi persistida em estado intermédio. O evento foi publicado no Kafka mas o consumidor responsável por o processar não está disponível. Quando o consumidor for restaurado, irá retomar a partir do *offset* em que parou e processar o evento normalmente, **mas só se** o sistema tiver sido desenhado para tolerar entrega *at-least-once* (o que exige idempotência) e mensagens com tempo de vida indeterminado (o que exige *dead-letter queue* para casos em que o processamento falhe persistentemente). Sem estes padrões, que não estão implementados neste protótipo, o estado inconsistente persiste indefinidamente, o sistema fica num estado *zombie*.

A hipótese H3 confirma-se. A consistência transacional, oferecida automaticamente pelo motor JPA no monólito, tem de ser reconquistada nos microserviços com esforço adicional. O ganho de desacoplamento temporal proporcionado pela comunicação assíncrona é pago em complexidade de consistência: padrões como Saga, idempotência e *dead-letter queues* deixam de ser sofisticacões opcionais e passam a ser exigências resultantes da própria arquitetura.

Implicações para Sistemas de Produção. Num sistema em produção, este comportamento tem consequências graves: o cliente recebeu confirmação de sucesso (HTTP 202), mas a encomenda nunca será processada sem intervenção manual. Se o *inventory-service* for reativado sem idempotência implementada, poderá processar eventos duplicados. Em cenários de alta carga, o lag no Kafka pode acumular centenas de encomendas em estado zombie antes de ser detetado. A diferença fundamental face ao monólito é conceptual: o monólito oferece consistência forte, estado sempre coerente; os microserviços, na forma básica, oferecem disponibilidade aparente mas consistência eventual com risco de estados intermédios permanentes. Este trade-off é descrito pelo teorema CAP e pelo modelo BASE (Basically Available, Soft state, Eventually consistent).

Padrões de Mitigação. A literatura de microserviços identifica três padrões principais para resolver este problema. O padrão Saga divide a transação em passos compensáveis: cada serviço executa a sua parte e, em caso de falha, executa uma ação de compensação, por exemplo, cancelar a encomenda e libertar o stock se o *inventory-service* não responder dentro de um timeout. O padrão Outbox garante atomicidade entre a escrita em base de dados e a publicação no Kafka: o Order Service escreve o evento numa tabela outbox na mesma transação JPA e um processo separado publica, eliminando a janela de inconsistência. O padrão Idempotent Consumer garante que mensagens duplicadas resultantes de retry não produzem efeitos secundários. Nenhum destes padrões está implementado no protótipo, o objetivo era demonstrar o comportamento base sem mitigações. A confirmação da H3 reforça que a decomposição em microserviços, sem estes padrões, introduz riscos de inconsistência que não existem na arquitetura monolítica.

6.5 H4 - Recursos em idle

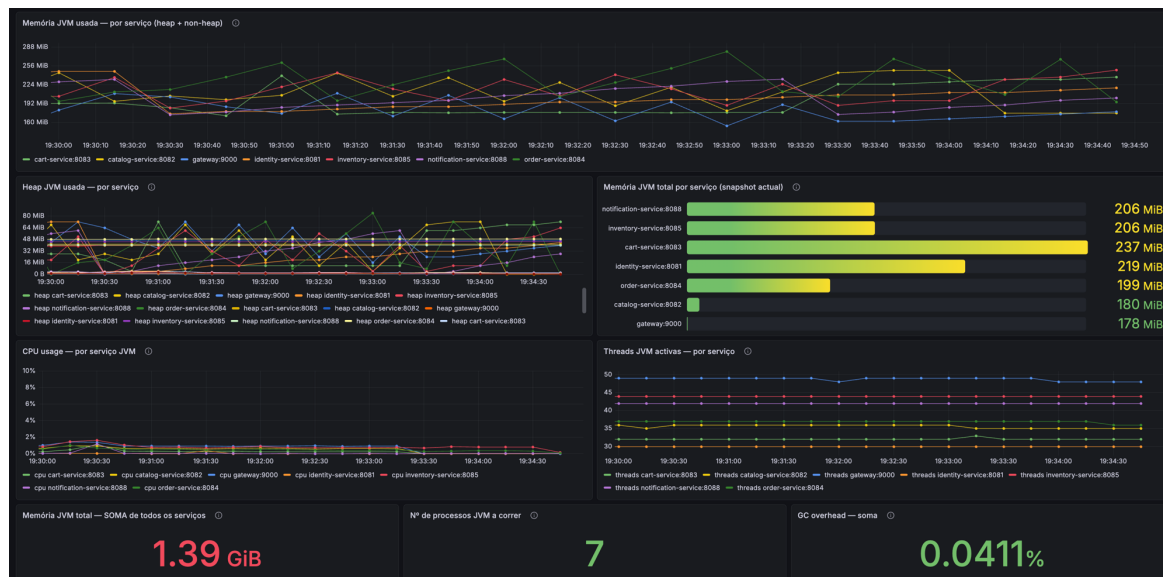
Objetivo da Experiência. Verificar se o consumo de memória RAM nos microserviços é dominado por um overhead fixo, independente do volume de tráfego, e se esse custo constitui uma penalização estrutural permanente. A hipótese H4 prevê que o custo de memória seja aproximadamente proporcional ao número de processos JVM e exista mesmo na ausência de utilizadores.

Método. Captura de RAM e CPU com docker stats em estado ocioso (sem tráfego k6), após um período de estabilização de 5 minutos a contar do arranque. Foram realizadas 24 amostras com intervalos de 5 segundos, em 3 execuções independentes, reportando-se a mediana. A infraestrutura de observabilidade (Jaeger, Prometheus, Grafana, Kafdrop) foi excluída do rácio principal de comparação, dado que não integra a aplicação em si, sendo reportada separadamente para referência.

Arquitetura	Componente	RAM (MB)
Monólito	monolith-app (1 JVM)	827
	monolith-db (PostgreSQL)	67
	monolith-redis	8
	TOTAL aplicação	912
Microserviços	api-gateway	426
	identity-service	360
	catalog-service	388
	cart-service	411
	order-service	441
	inventory-service	401
	notification-service	345
	6 × PostgreSQL	280
	Kafka + Zookeeper	713

Arquitetura	Componente	RAM (MB)
	TOTAL aplicação	3766
	(Jaeger + Prometheus + Grafana + Kafdrop — observabilidade, excluída do rácio)	705

Rácio RAM total micro / mono (aplicação): $3766 / 912 = 4.1\times$ Rácio incluindo observabilidade: $4471 / 912 = 4.9\times$



Análise e Interpretação dos Dados. O consumo de memória dos microserviços é 4,1 vezes superior ao do monólito em condições de inatividade. O overhead decompõe-se em dois factores estruturalmente independentes. O primeiro é o custo por processo JVM: cada um dos 7 serviços consome em mediana 396 MB de RAM em estado ocioso, totalizando 2772 MB exclusivamente para a camada aplicacional. O segundo factor, frequentemente subestimado, é o custo da infraestrutura de mensageria: Kafka e Zookeeper consomem em conjunto 713 MB em idle, valor superior ao consumo total do monólito (912 MB), inteiramente independente do número de mensagens processadas.

O facto de a variação de RAM entre estado ocioso e estado sob carga ser desprezível em ambas as arquitecturas confirma que este custo é estrutural e não escalável com o uso. Manter a infraestrutura de microserviços sem tráfego algum implica um custo de memória 4,1 vezes superior ao do monólito nas mesmas condições. A observabilidade (Jaeger, Prometheus, Grafana, Kafdrop = ~705 MB) acrescenta um custo adicional que em produção seria partilhado por múltiplas aplicações, diluindo o impacto por serviço.

Conclusão. A hipótese H4 é confirmada. O custo de memória é aproximadamente proporcional ao número de processos JVM, é estrutural e não escalável com o uso, validando integralmente a hipótese de overhead fixo dominante.

7 DISCUSSÃO

Os resultados experimentais permitem interpretar a decisão de decomposição à luz dos quatro atributos de qualidade considerados no enunciado — desempenho, disponibilidade, deployability e custo operacional — e dos trade-offs que lhes são inerentes.

Deployability. A decomposição resolve o problema central identificado na arquitectura AS-IS: qualquer alteração pontual num módulo obrigava ao redeployment total da aplicação, interrompendo temporariamente todos os domínios. A arquitectura TO-BE permite, em princípio, evoluir cada serviço de forma independente. Contudo, os dados de H1 revelam que esta independência é parcial: o acoplamento estático foi substituído por 13 vínculos dinâmicos de runtime — 5 contratos de API REST e 5 esquemas de mensagens Kafka —, cuja alteração continua a exigir coordenação entre serviços. A natureza do acoplamento mudou de forma relevante: enquanto no monólito qualquer módulo podia aceder ao estado interno de qualquer domínio sem restrição (o ShoppingCart guardava a entidade JPA Product completa, incluindo preço, peso, desconto, categoria e marca), nos microserviços cada serviço é o único proprietário dos seus dados. A deployability melhora efectivamente, mas traz consigo o ónus da governação activa de contratos.

Desempenho. Os resultados de H2 evidenciam que o impacto sobre a latência é assimétrico e dependente do contexto de execução. Em fluxos simples (T1), o overhead de +4,8 ms é marginal e não penaliza o utilizador de forma perceptível. Em fluxos transacionais (T3), o overhead mantém-se praticamente igual (+5,0 ms), dado que o custo marginal por salto adicional em localhost é da ordem de 1 ms. Contudo, o trace Jaeger revela que 59% do tempo de processamento do checkout é já consumido em chamadas de rede em localhost — um sinal de fragilidade latente que se tornaria dominante em produção, onde cada salto REST custaria entre 10 e 50 ms.

O problema identificado em H2 — chamadas síncronas em cascata no checkout — tem soluções conhecidas na literatura. A desnormalização de dados no order-service (persistir o preço no momento da compra, eliminando a chamada ao catalog-service em cada checkout) é a abordagem mais simples e adequada ao caso de uso. Em alternativa, a composição no gateway (padrão Backend for Frontend) ou a introdução de cache local no order-service com TTL curto reduziriam os saltos síncronos de 3 para 1–2 no caminho feliz.

Disponibilidade. O isolamento de falhas é o argumento mais robusto a favor da decomposição. O circuit breaker activo no API Gateway garante que uma falha num serviço não se propaga aos domínios restantes. Este benefício foi confirmado empiricamente em H3: durante o cenário de falha, os seis serviços restantes permaneceram operacionais e o gateway devolveu respostas normais para todos os outros fluxos. No monólito, qualquer degradação num módulo propagava-se imediatamente a toda a aplicação — incluindo operações completamente independentes como a consulta do catálogo.

Contudo, o mesmo cenário expõe o reverso desta disponibilidade aparente. O sistema aceitou o pedido de checkout com HTTP 202, transmitindo ao cliente uma confirmação de sucesso, quando na realidade a encomenda ficou num estado intermédio e o carrinho foi destruído irreversivelmente. Esta é a expressão concreta do trade-off descrito pelo teorema CAP: ao privilegiar a disponibilidade e o desacoplamento temporal, abriu-se uma janela de inconsistência que só pode ser fechada com padrões explícitos de compensação — Saga para as acções de rollback, idempotência para tolerar entrega *at-least-once* e dead-letter queue para mensagens de processamento persistentemente falhadas.

Custo operacional. O rácio de 4,1× em estado ocioso confirma que o custo da decomposição é estrutural e permanente. Dois factores dominam este overhead. O primeiro é o custo por processo JVM ($\sim 396 \text{ MB} \times 7 = 2772 \text{ MB}$). O segundo, frequentemente subestimado, é o custo da infraestrutura de mensageria: Kafka e Zookeeper consomem 713 MB em repouso — mais do que o monólito completo — sem processar uma única mensagem. Este resultado tem uma implicação directa para a decisão de adoptar Kafka: para o volume de tráfego deste protótipo, o barramento assíncrono não é amortizado pelo benefício de desacoplamento. Em produção com elevado volume de eventos e infraestrutura partilhada, este custo seria

diluído; no contexto actual do XCommerce, o padrão Outbox com polling seria uma alternativa de menor custo operacional para os mesmos objectivos de desacoplamento.

Trade-offs consolidados.

Atributo	Monólito	Microserviços	Observação
Deployability	Deployment acoplado	Deployment independente por domínio	Melhoria real, mas exige governação de contratos
Desempenho — leitura (T1)	8,9 ms p95	13,7 ms p95 (+54%)	Overhead marginal em termos absolutos
Desempenho — checkout (T3)	12,0 ms p95	17,0 ms p95 (+42%)	Em localhost; gap substancialmente maior em produção
Disponibilidade — isolamento	Falha propaga-se a todos os domínios	Falha contida ao domínio — circuit breaker activo	Benefício real e confirmado empiricamente
Consistência falha parcial	Rollback ACID automático	Estado zombie sem Saga	Regressão significativa sem padrões de compensação
Custo — Idle (RAM)	912 MB	3766 MB (4,1×)	Custo estrutural permanente, independente do tráfego
Custo — escalonamento selectivo	Impossível	Possível por domínio	Benefício potencial não exercitado no protótipo

8 CONCLUSÕES

8.1 Respostas às Questões de Investigação

QI1 — A decomposição elimina ou redistribui o acoplamento?

A decomposição redistribui o acoplamento sem o reduzir. O total de pontos de contacto cross-domain aumentou de 6 para 13 (+117%): as dependências estáticas compile-time foram substituídas por dependências dinâmicas runtime de maior volume e menor visibilidade. A vantagem não é a eliminação do acoplamento, mas a sua explicitação em contratos declarados e versionáveis — à custa de transferir a responsabilidade pela sua integridade do compilador para a equipa.

QI2 — A que custo em latência se paga a decomposição?

O custo é moderado em ambiente localhost, dominado pelo custo fixo do API Gateway (~5 ms), e não cresce proporcionalmente com o número de saltos nas condições laboratoriais deste estudo. Contudo, o trace Jaeger revela que 59% do tempo de processamento do checkout é já consumido em chamadas de rede, o que constitui um indicador de que,

em produção, o padrão de orquestração síncrona seria o factor dominante de degradação. O custo de memória em estado ocioso é de $4,1\times$, estrutural e independente do tráfego.

QI3 — A comunicação assíncrona melhora ou degrada a consistência sob falha parcial?

Na ausência de mecanismos de compensação, degrada. O cenário de estado zombie demonstrou empiricamente que a garantia ACID providenciada automaticamente pelo monólito tem de ser reconquistada nos microserviços através da implementação explícita de Saga, idempotência e dead-letter queue. A ausência destes padrões não é uma limitação do protótipo que desaparece em produção — é uma dívida técnica que cresce com o volume de tráfego e a frequência de falhas parciais.

QI4 — Qual o custo operacional em repouso?

O overhead fixo em estado ocioso é de $4,1\times$, dominado pelo custo por JVM e pela infraestrutura de mensageria. Existe independentemente da presença de utilizadores e constitui o custo de entrada da decomposição.

8.2 Arquitetura Principal

A decomposição em microserviços resolve os problemas que motivaram a transição — acoplamento estrutural de deployment, propagação irrestrita de falhas e impossibilidade de escalar domínios independentemente — mas cada resolução introduz um custo correspondente: o deployment independente exige controlo de API; o isolamento de falhas exige circuit breakers e padrões de compensação; a escala selectiva exige infraestrutura de mensageria que tem custo fixo independente do uso.

O resultado central deste estudo é que esses custos **não são uniformes**: o overhead de latência em localhost é marginal e relativamente constante (4-5ms por pedido independentemente dos hops); o overhead de consistência é crítico e exige trabalho activo (Saga, idempotência, DLQ); e o overhead operacional é significativo em repouso mas amortizável em produção.

A transição está incompleta sem Saga, idempotência e dead-letter queue — não como sofisticação opcional, mas como requisitos directos da decomposição assíncrona que eliminou as garantias ACID do monólito.

8.3 Limitações

- **Sem Saga compensatória**: a consistência eventual não é garantida em caso de falha parcial no checkout — estado zombie confirmado empiricamente.
- **Sem idempotência**: mensagens Kafka duplicadas podem resultar em decrementos de stock múltiplos para a mesma encomenda.
- **Sem dead-letter queue**: mensagens não processadas são perdidas silenciosamente após falha do consumer.
- **Ambiente localhost**: o overhead de rede entre serviços é subestimado (latência real em produção seria $10\text{-}50\times$ superior), o que favorece artificialmente os microserviços nas métricas de latência.
- **Dataset sintético e carga baixa (10 VUs)**: os resultados reflectem comportamento em carga moderada — sob carga elevada, o isolamento de falhas dos microserviços seria uma vantagem mais visível.
- **Kafka subdimensionado**: em produção, Kafka seria partilhado por múltiplos sistemas, diluindo o custo operacional de 713 MB por mais do que uma aplicação.

8.4 Extensões Futuras

1. **Implementar Saga por coreografia** para o fluxo de checkout e repetir o cenário H3 para verificar se o estado zombie é eliminado e o carrinho reposto em caso de falha do inventory-service.
2. **Adicionar idempotência** ao inventory-service (campo processed_order_id único) e repetir o cenário de entrega duplicada para verificar a eliminação de decrementos duplos.
3. **Testar em rede real** (serviços em VMs separadas ou Kubernetes) para obter métricas de latência representativas de produção, esperado rácio T3/T1 significativamente superior ao observado em localhost.
4. **Avaliar scaling selectivo** do catalog-service sob carga elevada (500+ VUs) para verificar se o isolamento por domínio permite escalar só o catálogo sem replicar a lógica de checkout.
5. **Substituir Kafka por outbox pattern com polling** para os eventos de notificação, e comparar o custo operacional em idle. potencial redução de 713 MB sem perda de desacoplamento para o caso de uso de notificações.